50 Full-Stack Projects with Clear Requirements

> Complete Frontend & Backend Specifications | 10-Day Projects Each

---

Project Difficulty Legend

Symbol	Difficulty	Time Estimate
🟢	Beginner	5-7 days
🟡	Intermediate	7-10 days
🔴	Advanced	10-14 days

Project 1: Task Manager (Todo App Pro) 🟢

Description

A feature-rich todo application with categories, priorities, and due dates.

Features

- ✅ Create, Read, Update, Delete tasks
- ✅ Task categories (Work, Personal, Shopping, etc.)
- ✅ Priority levels (Low, Medium, High, Urgent)
- ✅ Due dates with overdue highlighting
- ✅ Mark complete/incomplete
- ✅ Filter by status, category, priority
- ✅ Search tasks
- ✅ Drag and drop reordering
- ✅ Dark/Light theme

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
State: Zustand
Forms: react-hook-form + zod
Drag & Drop: @dnd-kit/core
Icons: lucide-react
Date: date-fns
```

Backend API (Node + Express + MongoDB)

Method	Endpoint	Description
POST	`/api/auth/register`	Register user
POST	`/api/auth/login`	Login user
GET	`/api/tasks`	Get all tasks (with filters)
POST	`/api/tasks`	Create task
GET	`/api/tasks/:id`	Get single task
PUT	`/api/tasks/:id`	Update task
DELETE	`/api/tasks/:id`	Delete task
PATCH	`/api/tasks/:id/toggle`	Toggle complete status
PATCH	`/api/tasks/reorder`	Update task order
GET	`/api/categories`	Get categories
POST	`/api/categories`	Create category

MongoDB Schema

```
// User
{ name, email, password, createdAt }

// Task
{ userId, title, description, category, priority, dueDate,
  completed, order, createdAt, updatedAt }

// Category
{ userId, name, color, icon }
```

Project 2: Expense Tracker 🟢

Description

Track daily expenses with categories, budgets, and visual reports.

Features

- ✅ Add income and expenses
- ✅ Categorize transactions
- ✅ Monthly budget setting
- ✅ Dashboard with charts
- ✅ Filter by date range
- ✅ Export to CSV
- ✅ Recurring transactions
- ✅ Budget alerts

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Charts: recharts
State: Zustand
Forms: react-hook-form + zod
Date Picker: react-day-picker
Export: file-saver
```

Backend API

Method	Endpoint	Description
POST	`/api/auth/register`	Register
POST	`/api/auth/login`	Login
GET	`/api/transactions`	Get transactions (filters: date, category, type)
POST	`/api/transactions`	Create transaction
PUT	`/api/transactions/:id`	Update transaction
DELETE	`/api/transactions/:id`	Delete transaction

Method	Endpoint	Description
GET	`/api/transactions/summary`	Get monthly summary
GET	`/api/budgets`	Get budgets
POST	`/api/budgets`	Set budget for category
PUT	`/api/budgets/:id`	Update budget
GET	`/api/categories`	Get expense categories

MongoDB Schema

```
// Transaction
{ userId, type: 'income'|'expense', amount, category,
  description, date, isRecurring, recurringInterval }

// Budget
{ userId, category, amount, month, year }

// Category
{ userId, name, type, icon, color }
```

Project 3: Note Taking App 🟢

Description

Markdown-based note app with folders, tags, and search.

Features

- ✅ Create/edit notes with Markdown
- ✅ Organize in folders
- ✅ Tag notes
- ✅ Full-text search
- ✅ Pin important notes
- ✅ Archive notes
- ✅ Trash with restore
- ✅ Auto-save
- ✅ Note sharing (public link)

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Markdown: @uiw/react-md-editor
State: Zustand
Search: use-debounce
Icons: lucide-react
```

Backend API

Method	Endpoint	Description
GET	`/api/notes`	Get notes (filters: folder, tag, search)
POST	`/api/notes`	Create note
GET	`/api/notes/:id`	Get note
PUT	`/api/notes/:id`	Update note
DELETE	`/api/notes/:id`	Move to trash
PATCH	`/api/notes/:id/pin`	Toggle pin
PATCH	`/api/notes/:id/archive`	Archive note
POST	`/api/notes/:id/share`	Generate share link
GET	`/api/public/notes/:shareId`	Get shared note (public)
GET	`/api/folders`	Get folders
POST	`/api/folders`	Create folder
PUT	`/api/folders/:id`	Rename folder
DELETE	`/api/folders/:id`	Delete folder
GET	`/api/tags`	Get all tags
DELETE	`/api/trash`	Empty trash
PATCH	`/api/trash/:id/restore`	Restore from trash

MongoDB Schema

```
// Note
{ userId, title, content, folderId, tags: [], isPinned,
  isArchived, isTrashed, shareId, isPublic, createdAt, updatedAt }

// Folder
{ userId, name, parentId, order }
```

Project 4: Habit Tracker 🟢

Description

Track daily habits and build streaks.

Features

- ✅ Create habits with frequency (daily, weekly, specific days)
- ✅ Mark habit complete for today
- ✅ View habit calendar (GitHub-style heatmap)
- ✅ Streak tracking
- ✅ Statistics and insights
- ✅ Habit categories
- ✅ Reminders (time-based)
- ✅ Weekly/monthly progress reports

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Calendar Heatmap: react-calendar-heatmap
Charts: recharts
State: Zustand
Date: date-fns
Notifications: react-toastify
```

Backend API

Method	Endpoint	Description

Method	Endpoint	Description
GET	`/api/habits`	Get all habits
POST	`/api/habits`	Create habit
PUT	`/api/habits/:id`	Update habit
DELETE	`/api/habits/:id`	Delete habit
POST	`/api/habits/:id/complete`	Mark complete for date
DELETE	`/api/habits/:id/complete`	Unmark complete
GET	`/api/habits/:id/history`	Get completion history
GET	`/api/habits/:id/stats`	Get streak and stats
GET	`/api/stats/weekly`	Weekly overview
GET	`/api/stats/monthly`	Monthly overview

MongoDB Schema

```
// Habit
{ userId, name, description, category, frequency,
  targetDays: [], color, icon, reminderTime, createdAt }

// HabitLog
{ habitId, userId, date, completed: true }
```

Project 5: Recipe Manager 🟡

Description

Store, organize, and discover recipes.

Features

- ✅ Add recipes with ingredients and steps
- ✅ Upload recipe images
- ✅ Categorize (Breakfast, Lunch, Dinner, Dessert)
- ✅ Search and filter
- ✅ Favorite recipes

- ✅ Meal planning (weekly calendar)
- ✅ Shopping list generation
- ✅ Nutritional info (optional)
- ✅ Recipe scaling (servings)
- ✅ Share recipes

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Image Upload: react-dropzone
State: Zustand + @tanstack/react-query
Calendar: react-big-calendar
Forms: react-hook-form + zod
```

Backend API

Method	Endpoint	Description
GET	`/api/recipes`	Get recipes (filters)
POST	`/api/recipes`	Create recipe
GET	`/api/recipes/:id`	Get recipe
PUT	`/api/recipes/:id`	Update recipe
DELETE	`/api/recipes/:id`	Delete recipe
POST	`/api/recipes/:id/favorite`	Toggle favorite
POST	`/api/upload/image`	Upload recipe image
GET	`/api/meal-plans`	Get meal plan for week
POST	`/api/meal-plans`	Add recipe to meal plan
DELETE	`/api/meal-plans/:id`	Remove from plan
GET	`/api/shopping-list`	Generate shopping list
POST	`/api/shopping-list/item`	Add item to list
PATCH	`/api/shopping-list/item/:id`	Toggle item bought

MongoDB Schema

```
// Recipe
{ userId, title, description, image, category, cuisine,
  prepTime, cookTime, servings,
  ingredients: [{ name, amount, unit }],
  instructions: [{ step, description }],
  tags: [], isFavorite, isPublic, createdAt }

// MealPlan
{ userId, recipeId, date, mealType: 'breakfast'|'lunch'|'dinner' }

// ShoppingItem
{ userId, name, amount, unit, checked, recipeId }
```

Project 6: Bookmark Manager 🟢

Description

Save, organize, and access bookmarks across devices.

Features

- ✅ Add bookmarks with auto-fetch title/favicon
- ✅ Organize in collections/folders
- ✅ Tag bookmarks
- ✅ Search bookmarks
- ✅ Browser extension integration
- ✅ Import/export bookmarks
- ✅ Quick access favorites
- ✅ Reading list (read later)

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
State: Zustand + @tanstack/react-query
DnD: @dnd-kit/core
Icons: lucide-react
```

Backend API

Method	Endpoint	Description
GET	`/api/bookmarks`	Get bookmarks (filters)
POST	`/api/bookmarks`	Add bookmark
PUT	`/api/bookmarks/:id`	Update bookmark
DELETE	`/api/bookmarks/:id`	Delete bookmark
POST	`/api/bookmarks/:id/favorite`	Toggle favorite
POST	`/api/bookmarks/import`	Import from file
GET	`/api/bookmarks/export`	Export bookmarks
GET	`/api/collections`	Get collections
POST	`/api/collections`	Create collection
PUT	`/api/collections/:id`	Update collection
DELETE	`/api/collections/:id`	Delete collection
GET	`/api/tags`	Get all tags
POST	`/api/fetch-metadata`	Fetch URL metadata

MongoDB Schema

```
// Bookmark
{ userId, url, title, description, favicon, image,
  collectionId, tags: [], isFavorite, isReadLater,
  isRead, createdAt }

// Collection
{ userId, name, icon, color, parentId }
```

Project 7: Personal Blog Platform 🟡

Description

Create and manage your own blog with admin panel.

Features

- ✅ Write posts with rich text editor
- ✅ Draft and publish workflow
- ✅ Categories and tags
- ✅ Featured images
- ✅ SEO settings (meta title, description)
- ✅ Comments system
- ✅ View count tracking
- ✅ Related posts
- ✅ RSS feed
- ✅ Social sharing

Frontend Stack

```
Framework: Next.js 14
UI Library: shadcn/ui
Styling: Tailwind CSS
Editor: tiptap
State: Zustand
Data Fetching: @tanstack/react-query
SEO: next-seo
Syntax Highlight: prism-react-renderer
```

Backend API

Method	Endpoint	Description
GET	`/api/posts`	Get posts (public, with pagination)
GET	`/api/posts/:slug`	Get post by slug
POST	`/api/admin/posts`	Create post
PUT	`/api/admin/posts/:id`	Update post
DELETE	`/api/admin/posts/:id`	Delete post
PATCH	`/api/admin/posts/:id/publish`	Publish/unpublish
POST	`/api/upload/image`	Upload image
GET	`/api/posts/:id/comments`	Get comments
POST	`/api/posts/:id/comments`	Add comment

Method	Endpoint	Description
DELETE	`/api/comments/:id`	Delete comment
GET	`/api/categories`	Get categories
POST	`/api/admin/categories`	Create category
GET	`/api/tags`	Get tags
GET	`/api/feed/rss`	RSS feed

MongoDB Schema

```
// Post
{ userId, title, slug, content, excerpt, featuredImage,
  category, tags: [], status: 'draft'|'published',
  seoTitle, seoDescription, viewCount, publishedAt, createdAt }

// Comment
{ postId, name, email, content, isApproved, parentId, createdAt }
```

Project 8: Job Application Tracker 🟢

Description

Track job applications, interviews, and follow-ups.

Features

- ✅ Add job applications
- ✅ Status tracking (Applied, Interview, Offer, Rejected)
- ✅ Kanban board view
- ✅ Calendar view for interviews
- ✅ Add notes per application
- ✅ Attach resume per application
- ✅ Contact information
- ✅ Salary tracking
- ✅ Statistics dashboard
- ✅ Reminders for follow-ups

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Kanban: @dnd-kit/core
Calendar: react-big-calendar
Charts: recharts
State: Zustand
Forms: react-hook-form + zod
```

Backend API

Method	Endpoint	Description
GET	`/api/applications`	Get applications
POST	`/api/applications`	Create application
GET	`/api/applications/:id`	Get application details
PUT	`/api/applications/:id`	Update application
DELETE	`/api/applications/:id`	Delete application
PATCH	`/api/applications/:id/status`	Update status
POST	`/api/applications/:id/notes`	Add note
GET	`/api/applications/:id/notes`	Get notes
DELETE	`/api/notes/:id`	Delete note
POST	`/api/applications/:id/interview`	Add interview
GET	`/api/interviews`	Get all interviews
GET	`/api/stats`	Get statistics
POST	`/api/upload/resume`	Upload resume

MongoDB Schema

```
// Application
{ userId, company, position, location, salary,
  jobUrl, status, appliedDate, resumeUrl,
  contactName, contactEmail, createdAt }
```

```
// Interview
{ applicationId, userId, date, type: 'phone'|'video'|'onsite',
  notes, status: 'scheduled'|'completed'|'cancelled' }

// Note
{ applicationId, content, createdAt }
```

Project 9: Workout Tracker 🟡

Description

Log workouts, track progress, and plan routines.

Features

- ✅ Create workout routines
- ✅ Exercise library (with GIFs/images)
- ✅ Log workouts (sets, reps, weight)
- ✅ Track personal records
- ✅ Progress charts
- ✅ Calendar view
- ✅ Timer for rest periods
- ✅ Workout templates
- ✅ Body measurements tracking

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Charts: recharts
Calendar: react-big-calendar
Timer: Custom hook
State: Zustand
Forms: react-hook-form
```

Backend API

Method	Endpoint	Description

Method	Endpoint	Description
GET	`/api/exercises`	Get exercise library
POST	`/api/exercises`	Add custom exercise
GET	`/api/routines`	Get workout routines
POST	`/api/routines`	Create routine
PUT	`/api/routines/:id`	Update routine
DELETE	`/api/routines/:id`	Delete routine
GET	`/api/workouts`	Get logged workouts
POST	`/api/workouts`	Log workout
GET	`/api/workouts/:id`	Get workout details
PUT	`/api/workouts/:id`	Update workout
GET	`/api/stats/exercise/:id`	Get exercise progress
GET	`/api/stats/personal-records`	Get PRs
GET	`/api/measurements`	Get body measurements
POST	`/api/measurements`	Log measurement

MongoDB Schema

```
// Exercise
{ name, category, muscleGroup, equipment, description,
  image, isCustom, userId }

// Routine
{ userId, name, exercises: [{ exerciseId, sets, reps, rest }] }

// Workout
{ userId, routineId, date, duration,
  exercises: [{ exerciseId, sets: [{ reps, weight }] }] }

// Measurement
{ userId, date, weight, bodyFat, chest, waist, hips, arms, thighs }
```

Project 10: Invoice Generator 🟡

Description

Create and send professional invoices.

Features

- ✅ Create invoices with line items
- ✅ Client management
- ✅ Tax calculation
- ✅ Multiple currencies
- ✅ Invoice templates
- ✅ PDF generation
- ✅ Email invoices
- ✅ Payment status tracking
- ✅ Recurring invoices
- ✅ Dashboard with revenue stats

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
PDF: @react-pdf/renderer
Charts: recharts
State: Zustand
Forms: react-hook-form + zod
Date: date-fns
```

Backend API

Method	Endpoint	Description
GET	`/api/invoices`	Get invoices
POST	`/api/invoices`	Create invoice
GET	`/api/invoices/:id`	Get invoice
PUT	`/api/invoices/:id`	Update invoice
DELETE	`/api/invoices/:id`	Delete invoice
PATCH	`/api/invoices/:id/status`	Update payment status

Method	Endpoint	Description
GET	`/api/invoices/:id/pdf`	Generate PDF
POST	`/api/invoices/:id/send`	Send via email
GET	`/api/clients`	Get clients
POST	`/api/clients`	Add client
PUT	`/api/clients/:id`	Update client
DELETE	`/api/clients/:id`	Delete client
GET	`/api/settings`	Get business settings
PUT	`/api/settings`	Update settings
GET	`/api/stats/revenue`	Revenue statistics

MongoDB Schema

```
// Invoice
{ userId, invoiceNumber, clientId, items: [{ description,
  quantity, rate, amount }], subtotal, tax, taxRate,
  total, currency, status: 'draft'|'sent'|'paid'|'overdue',
  dueDate, notes, createdAt }

// Client
{ userId, name, email, phone, address, company }

// Settings
{ userId, businessName, email, phone, address, logo,
  taxRate, currency, invoicePrefix }
```

Project 11: Poll & Survey Creator 🟡

Description

Create polls and surveys, collect responses.

Features

- ✅ Create polls (single/multiple choice)

- ✅ Create surveys (multiple questions)
- ✅ Question types: text, radio, checkbox, rating, scale
- ✅ Anonymous responses option
- ✅ Response limits
- ✅ Expiry date
- ✅ Results visualization
- ✅ Export results to CSV
- ✅ Share via link
- ✅ Embed code

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Charts: recharts
DnD: @dnd-kit/core
State: Zustand
Forms: react-hook-form
Export: file-saver
```

Backend API

Method	Endpoint	Description
GET	`/api/polls`	Get user's polls
POST	`/api/polls`	Create poll
GET	`/api/polls/:id`	Get poll (for creator)
PUT	`/api/polls/:id`	Update poll
DELETE	`/api/polls/:id`	Delete poll
PATCH	`/api/polls/:id/close`	Close poll
GET	`/api/public/polls/:shareId`	Get poll (public)
POST	`/api/public/polls/:shareId/vote`	Submit vote
GET	`/api/polls/:id/results`	Get results
GET	`/api/polls/:id/export`	Export CSV
GET	`/api/surveys`	Get surveys
POST	`/api/surveys`	Create survey

Method	Endpoint	Description
GET	`/api/public/surveys/:shareId`	Get survey (public)
POST	`/api/public/surveys/:shareId/submit`	Submit response

MongoDB Schema

```
// Poll
{ userId, title, description, options: [{ text, votes }],
  isMultipleChoice, isAnonymous, maxResponses, expiresAt,
  shareId, isClosed, createdAt }

// PollVote
{ pollId visitorId/userId, selectedOptions: [], createdAt }

// Survey
{ userId, title, description,
  questions: [{ type, text, options, required }],
  shareId, isAnonymous, expiresAt, createdAt }

// SurveyResponse
{ surveyId, answers: [{ questionIndex, answer }], createdAt }
```

Project 12: Event Countdown App 🟢

Description

Create countdowns for important events.

Features

- ✅ Create event with date/time
- ✅ Live countdown (days, hours, minutes, seconds)
- ✅ Categories (Birthday, Holiday, Wedding, etc.)
- ✅ Custom backgrounds/themes
- ✅ Share countdown page
- ✅ Recurring events (yearly birthday)
- ✅ Widget/embed code
- ✅ Email notification before event

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Animation: framer-motion
Date: date-fns
State: Zustand
```

Backend API

Method	Endpoint	Description
GET	`/api/events`	Get user's events
POST	`/api/events`	Create event
GET	`/api/events/:id`	Get event
PUT	`/api/events/:id`	Update event
DELETE	`/api/events/:id`	Delete event
GET	`/api/public/events/:shareId`	Get shared event
POST	`/api/events/:id/notification`	Set notification
GET	`/api/upcoming`	Get upcoming events

MongoDB Schema

```
// Event
{ userId, title, description, eventDate, endDate,
  category, isRecurring, recurringType, theme,
  backgroundImage, shareId, isPublic, createdAt }

// Notification
{ eventId, userId, email, notifyBefore: Number, sent: Boolean }
```

Project 13: Link Shortener 🟢

Description

Shorten URLs with analytics tracking.

Features

- ✅ Shorten long URLs
- ✅ Custom short codes
- ✅ QR code generation
- ✅ Click analytics
- ✅ Geographic data
- ✅ Device/browser stats
- ✅ Link expiration
- ✅ Password protection
- ✅ Bulk shortening

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
QR Code: qrcode.react
Charts: recharts
State: Zustand
Copy: react-copy-to-clipboard
```

Backend API

Method	Endpoint	Description
GET	`/api/links`	Get user's links
POST	`/api/links`	Create short link
GET	`/api/links/:id`	Get link details
PUT	`/api/links/:id`	Update link
DELETE	`/api/links/:id`	Delete link
GET	`/api/links/:id/stats`	Get analytics
POST	`/api/links/bulk`	Bulk create
GET	`/api/links/:id/qr`	Get QR code
GET	`/:shortCode`	Redirect (public)

Method	Endpoint	Description
POST	`/:shortCode/verify`	Verify password

MongoDB Schema

```
// Link
{ userId, originalUrl, shortCode, customCode,
  password, expiresAt, isActive, createdAt }

// Click
{ linkId, timestamp, ip, country, city,
  device, browser, os, referrer }
```

Project 14: Feedback Widget System 🟡

Description

Embeddable feedback widget for websites.

Features

- ✅ Create feedback widget
- ✅ Customizable appearance
- ✅ Multiple feedback types (Bug, Feature, General)
- ✅ Screenshot capture
- ✅ User identification
- ✅ Feedback dashboard
- ✅ Status tracking
- ✅ Respond to feedback
- ✅ Export feedback
- ✅ Embed script generation

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Screenshot: html2canvas
```

```
State: Zustand + @tanstack/react-query
Forms: react-hook-form
```

Backend API

Method	Endpoint	Description
GET	`/api/projects`	Get projects
POST	`/api/projects`	Create project
PUT	`/api/projects/:id`	Update project
DELETE	`/api/projects/:id`	Delete project
GET	`/api/projects/:id/script`	Get embed script
GET	`/api/projects/:id/feedback`	Get all feedback
POST	`/api/public/feedback`	Submit feedback (public)
GET	`/api/feedback/:id`	Get feedback details
PATCH	`/api/feedback/:id/status`	Update status
POST	`/api/feedback/:id/respond`	Respond to feedback
POST	`/api/upload/screenshot`	Upload screenshot

MongoDB Schema

```
// Project
{ userId, name, domain, widgetSettings: { color, position,
  types: [] }, apiKey, createdAt }

// Feedback
{ projectId, type, message, screenshot, userEmail,
  userName, pageUrl, browser, device,
  status: 'new'|'in-progress'|'resolved'|'closed',
  response, createdAt }
```

Project 15: Pomodoro Timer App 🟢

Description

Productivity timer with task integration.

Features

- ✅ Configurable timer (25/5/15 minutes)
- ✅ Auto-start breaks
- ✅ Task list integration
- ✅ Daily/weekly statistics
- ✅ Focus sounds (rain, café, etc.)
- ✅ Notifications
- ✅ Desktop notifications
- ✅ Streak tracking
- ✅ Goals (pomodoros per day)

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Audio: howler.js
Charts: recharts
State: Zustand
Timer: Custom hook
Notifications: Web Notifications API
```

Backend API

Method	Endpoint	Description
GET	`/api/tasks`	Get tasks
POST	`/api/tasks`	Create task
PUT	`/api/tasks/:id`	Update task
DELETE	`/api/tasks/:id`	Delete task
POST	`/api/pomodoros`	Log completed pomodoro
GET	`/api/pomodoros/today`	Get today's pomodoros
GET	`/api/stats/daily`	Daily stats
GET	`/api/stats/weekly`	Weekly stats
GET	`/api/settings`	Get timer settings

Method	Endpoint	Description
PUT	`/api/settings`	Update settings
GET	`/api/goals`	Get goals
PUT	`/api/goals`	Update goals

MongoDB Schema

```
// Task
{ userId, title, estimatedPomodoros, completedPomodoros,
  isCompleted, order, createdAt }

// Pomodoro
{ userId, taskId, duration, type: 'work'|'break',
  completedAt }

// Settings
{ userId, workDuration, shortBreak, longBreak,
  longBreakInterval, autoStartBreaks, sound }

// Goal
{ userId, dailyTarget, weeklyTarget }
```

Project 16: Reading List / Book Tracker 🟢

Description

Track books you want to read, are reading, and have read.

Features

- ✅ Add books (search by title/ISBN)
- ✅ Status: Want to Read, Reading, Finished
- ✅ Reading progress (page/percentage)
- ✅ Star rating and reviews
- ✅ Shelves/categories
- ✅ Reading goals (books per year)
- ✅ Statistics
- ✅ Quotes collection

- ✅ Share reading list

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Book API: Open Library API (client-side)
Charts: recharts
State: Zustand
```

Backend API

Method	Endpoint	Description
GET	`/api/books`	Get user's books
POST	`/api/books`	Add book
GET	`/api/books/:id`	Get book details
PUT	`/api/books/:id`	Update book
DELETE	`/api/books/:id`	Remove book
PATCH	`/api/books/:id/progress`	Update progress
PATCH	`/api/books/:id/status`	Update status
POST	`/api/books/:id/review`	Add review
GET	`/api/shelves`	Get shelves
POST	`/api/shelves`	Create shelf
GET	`/api/quotes`	Get quotes
POST	`/api/quotes`	Add quote
GET	`/api/stats/yearly`	Yearly reading stats
GET	`/api/goals`	Get reading goals
PUT	`/api/goals`	Set goals

MongoDB Schema

```
// Book
{ userId, title, author, isbn, cover, pages, genre,
```

```
    shelfId, status, currentPage, rating, review,
    startedAt, finishedAt, createdAt }

  // Shelf
  { userId, name, isDefault }

  // Quote
  { userId, bookId, text, page, createdAt }

  // Goal
  { userId, year, target, completed }
```

Project 17: Kanban Project Board 🟡

Description

Trello-like project management board.

Features

- ✅ Multiple boards
- ✅ Customizable columns
- ✅ Drag and drop cards
- ✅ Card details (description, checklist, due date)
- ✅ Labels/tags
- ✅ Member assignment
- ✅ Comments on cards
- ✅ Activity log
- ✅ Card cover images
- ✅ Archive cards/lists

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
DnD: @dnd-kit/core + @dnd-kit/sortable
State: Zustand + @tanstack/react-query
Forms: react-hook-form
Rich Text: tiptap (for descriptions)
```

Backend API

Method	Endpoint	Description
GET	`/api/boards`	Get boards
POST	`/api/boards`	Create board
GET	`/api/boards/:id`	Get board with lists/cards
PUT	`/api/boards/:id`	Update board
DELETE	`/api/boards/:id`	Delete board
POST	`/api/boards/:id/lists`	Create list
PUT	`/api/lists/:id`	Update list
DELETE	`/api/lists/:id`	Delete list
PATCH	`/api/lists/reorder`	Reorder lists
POST	`/api/lists/:id/cards`	Create card
GET	`/api/cards/:id`	Get card details
PUT	`/api/cards/:id`	Update card
DELETE	`/api/cards/:id`	Delete card
PATCH	`/api/cards/move`	Move card
POST	`/api/cards/:id/comments`	Add comment
GET	`/api/cards/:id/activity`	Get activity

MongoDB Schema

```
// Board
{ userId, title, background, isArchived, createdAt }

// List
{ boardId, title, order, isArchived }

// Card
{ listId, boardId, title, description, cover,
  labels: [], dueDate, checklist: [{ text, checked }],
  assignees: [], order, isArchived, createdAt }

// Comment
```

```
  { cardId, userId, content, createdAt }

  // Activity
  { boardId, cardId, userId, action, details, createdAt }
```

Project 18: Contact Manager / CRM Lite 🟡

Description

Manage contacts with interaction tracking.

Features

- ✅ Add/edit contacts
- ✅ Contact groups/tags
- ✅ Search and filter
- ✅ Interaction log (call, email, meeting)
- ✅ Reminders/follow-ups
- ✅ Import from CSV
- ✅ Export contacts
- ✅ Duplicate detection
- ✅ Birthday reminders
- ✅ Quick actions (call, email)

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
State: Zustand + @tanstack/react-query
Forms: react-hook-form + zod
CSV: papaparse
Calendar: date-fns
```

Backend API

Method	Endpoint	Description
GET	`/api/contacts`	Get contacts (filters, search)

Method	Endpoint	Description
POST	`/api/contacts`	Create contact
GET	`/api/contacts/:id`	Get contact
PUT	`/api/contacts/:id`	Update contact
DELETE	`/api/contacts/:id`	Delete contact
POST	`/api/contacts/import`	Import CSV
GET	`/api/contacts/export`	Export contacts
GET	`/api/contacts/duplicates`	Find duplicates
POST	`/api/contacts/:id/interactions`	Log interaction
GET	`/api/contacts/:id/interactions`	Get interactions
GET	`/api/groups`	Get groups
POST	`/api/groups`	Create group
GET	`/api/reminders`	Get due reminders
POST	`/api/reminders`	Create reminder
GET	`/api/birthdays/upcoming`	Upcoming birthdays

MongoDB Schema

```
// Contact
{ userId, firstName, lastName, email, phone, company,
  position, address, birthday, notes, avatar,
  groups: [], tags: [], createdAt }

// Interaction
{ contactId, userId, type: 'call'|'email'|'meeting'|'note',
  summary, date, createdAt }

// Reminder
{ userId, contactId, title, date, isComplete }

// Group
{ userId, name, color }
```

Project 19: Quiz App 🟡

Description

Create and take quizzes with scoring.

Features

- ✅ Create quizzes with multiple question types
- ✅ Question types: MCQ, True/False, Short Answer
- ✅ Timer per quiz or per question
- ✅ Randomize questions/options
- ✅ Score calculation
- ✅ Review answers after completion
- ✅ Leaderboard
- ✅ Share quiz via link
- ✅ Quiz attempts limit
- ✅ Certificate generation

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
State: Zustand
Timer: Custom hook
Charts: recharts
PDF: @react-pdf/renderer
Animation: framer-motion
```

Backend API

Method	Endpoint	Description
GET	`/api/quizzes`	Get user's quizzes
POST	`/api/quizzes`	Create quiz
GET	`/api/quizzes/:id`	Get quiz (creator view)
PUT	`/api/quizzes/:id`	Update quiz
DELETE	`/api/quizzes/:id`	Delete quiz
GET	`/api/public/quizzes/:shareId`	Get quiz (public)

Method	Endpoint	Description
POST	`/api/public/quizzes/:shareId/start`	Start attempt
POST	`/api/attempts/:id/submit`	Submit answers
GET	`/api/attempts/:id/results`	Get results
GET	`/api/quizzes/:id/leaderboard`	Get leaderboard
GET	`/api/quizzes/:id/stats`	Quiz statistics
GET	`/api/attempts/:id/certificate`	Generate certificate

MongoDB Schema

```
// Quiz
{ userId, title, description, questions: [{
  type, question, options, correctAnswer, points, timeLimit }],
  totalPoints, duration, attemptsAllowed, isRandomized,
  shareId, isPublished, createdAt }

// Attempt
{ quizId, userId, name, email, answers: [{ questionIndex,
  answer, isCorrect, points }], score, totalPoints,
  startedAt, completedAt }
```

Project 20: File Sharing App 🟡

Description

Upload and share files with download links.

Features

- ✅ Drag and drop upload
- ✅ Multiple file upload
- ✅ Progress indicator
- ✅ Generate shareable links
- ✅ Password protected links
- ✅ Link expiration
- ✅ Download count tracking

- ✅ File preview (images, PDFs)
- ✅ Folder creation
- ✅ Storage quota tracking

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Upload: react-dropzone
State: Zustand
Progress: Custom
Preview: react-pdf, react-photo-view
```

Backend API

Method	Endpoint	Description
POST	`/api/upload`	Upload file(s)
GET	`/api/files`	Get user's files
GET	`/api/files/:id`	Get file info
DELETE	`/api/files/:id`	Delete file
POST	`/api/files/:id/share`	Create share link
GET	`/api/shares`	Get user's share links
DELETE	`/api/shares/:id`	Delete share link
GET	`/api/public/download/:shareId`	Download file
POST	`/api/public/download/:shareId/verify`	Verify password
GET	`/api/folders`	Get folders
POST	`/api/folders`	Create folder
PUT	`/api/folders/:id`	Update folder
DELETE	`/api/folders/:id`	Delete folder
GET	`/api/storage`	Get storage usage

MongoDB Schema

```
// File
{ userId, folderId, filename, originalName, mimetype,
  size, path, createdAt }

// Share
{ fileId, userId, shareId, password, expiresAt,
  downloadLimit, downloadCount, createdAt }

// Folder
{ userId, name, parentId }
```

Project 21: Appointment Scheduler 🟡

Description

Book appointments with available time slots.

Features

- ✅ Set availability (working hours)
- ✅ Define appointment types/duration
- ✅ Public booking page
- ✅ Calendar view of appointments
- ✅ Email confirmations
- ✅ Reschedule/cancel
- ✅ Buffer time between appointments
- ✅ Timezone support
- ✅ Recurring availability
- ✅ Meeting links integration

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Calendar: react-big-calendar
Date: date-fns-tz (timezone)
State: Zustand + @tanstack/react-query
Forms: react-hook-form + zod
```

Backend API

Method	Endpoint	Description
GET	`/api/appointments`	Get user's appointments
GET	`/api/appointments/:id`	Get appointment
PATCH	`/api/appointments/:id/cancel`	Cancel appointment
PATCH	`/api/appointments/:id/reschedule`	Reschedule
GET	`/api/availability`	Get availability settings
PUT	`/api/availability`	Update availability
GET	`/api/appointment-types`	Get appointment types
POST	`/api/appointment-types`	Create type
PUT	`/api/appointment-types/:id`	Update type
DELETE	`/api/appointment-types/:id`	Delete type
GET	`/api/public/book/:username`	Get booking page
GET	`/api/public/book/:username/slots`	Get available slots
POST	`/api/public/book/:username`	Book appointment

MongoDB Schema

```
// User (extend)
{ username, timezone, bookingUrl }

// Availability
{ userId, dayOfWeek, startTime, endTime, isAvailable }

// AppointmentType
{ userId, name, duration, description, color, bufferTime,
  meetingLink, isActive }

// Appointment
{ userId, typeId, clientName, clientEmail, clientPhone,
  date, startTime, endTime, status: 'confirmed'|'cancelled'|'completed',
  notes, createdAt }
```

Project 22: Social Media Dashboard 🟡

Description

Schedule and manage social media posts.

Features

- ✅ Create posts for multiple platforms
- ✅ Schedule posts for later
- ✅ Content calendar view
- ✅ Image upload and preview
- ✅ Draft posts
- ✅ Post analytics (mock)
- ✅ Hashtag suggestions
- ✅ Best time to post suggestions
- ✅ Bulk scheduling
- ✅ Content categories

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Calendar: react-big-calendar
Image: react-dropzone
State: Zustand + @tanstack/react-query
Rich Text: tiptap
```

Backend API

Method	Endpoint	Description
GET	`/api/posts`	Get posts (filters)
POST	`/api/posts`	Create post
GET	`/api/posts/:id`	Get post
PUT	`/api/posts/:id`	Update post
DELETE	`/api/posts/:id`	Delete post

Method	Endpoint	Description
PATCH	`/api/posts/:id/publish`	Publish now
POST	`/api/posts/bulk`	Bulk schedule
POST	`/api/upload/media`	Upload image/video
GET	`/api/accounts`	Get connected accounts
POST	`/api/accounts`	Add account (mock)
DELETE	`/api/accounts/:id`	Remove account
GET	`/api/analytics`	Get analytics (mock)
GET	`/api/hashtags/suggest`	Get hashtag suggestions
GET	`/api/best-times`	Get best posting times

MongoDB Schema

```
// Post
{ userId, content, media: [{ url, type }], platforms: [],
  scheduledAt, publishedAt, status: 'draft'|'scheduled'|'published'|'fail
  category, hashtags: [], createdAt }

// Account
{ userId, platform, username, accessToken, profileImage }

// Analytics
{ postId, platform, likes, comments, shares, reach, date }
```

Project 23: Flashcard Study App 🟢

Description

Create and study flashcards with spaced repetition.

Features

- ✅ Create flashcard decks
- ✅ Add cards (front/back)
- ✅ Study mode with flip animation

- ✅ Spaced repetition algorithm
- ✅ Mark cards: Easy, Medium, Hard
- ✅ Progress tracking
- ✅ Study statistics
- ✅ Import/export decks
- ✅ Share decks publicly
- ✅ Quiz mode

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Animation: framer-motion
State: Zustand
Charts: recharts
Markdown: react-markdown
```

Backend API

Method	Endpoint	Description
GET	`/api/decks`	Get user's decks
POST	`/api/decks`	Create deck
GET	`/api/decks/:id`	Get deck with cards
PUT	`/api/decks/:id`	Update deck
DELETE	`/api/decks/:id`	Delete deck
POST	`/api/decks/:id/share`	Make public
GET	`/api/public/decks`	Browse public decks
GET	`/api/public/decks/:id`	Get public deck
POST	`/api/public/decks/:id/clone`	Clone to library
POST	`/api/decks/:id/cards`	Add card
PUT	`/api/cards/:id`	Update card
DELETE	`/api/cards/:id`	Delete card
GET	`/api/decks/:id/study`	Get cards to study
POST	`/api/cards/:id/review`	Submit review

Method	Endpoint	Description
GET	`/api/stats`	Get study stats
POST	`/api/decks/import`	Import deck

MongoDB Schema

```
// Deck
{ userId, title, description, category, cardCount,
  isPublic, cloneCount, createdAt }

// Card
{ deckId, front, back, frontImage, backImage }

// CardProgress
{ cardId, userId, easeFactor, interval, dueDate,
  repetitions, lastReviewed }

// StudySession
{ deckId, userId, cardsStudied, correctCount,
  duration, completedAt }
```

Project 24: Weather Dashboard 🟢

Description

Display weather with forecasts and saved locations.

Features

- ✅ Search cities
- ✅ Current weather display
- ✅ 5-day forecast
- ✅ Hourly forecast
- ✅ Save favorite locations
- ✅ Default location
- ✅ Temperature unit toggle (°C/°F)
- ✅ Weather alerts
- ✅ Weather maps

- ✅ Geolocation support

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Weather API: OpenWeatherMap (client-side)
State: Zustand
Charts: recharts
Icons: Weather icons library
Geolocation: Browser API
```

Backend API

Method	Endpoint	Description
GET	`/api/locations`	Get saved locations
POST	`/api/locations`	Save location
DELETE	`/api/locations/:id`	Remove location
PATCH	`/api/locations/:id/default`	Set as default
GET	`/api/settings`	Get user settings
PUT	`/api/settings`	Update settings (units)
GET	`/api/alerts`	Get weather alerts
POST	`/api/alerts`	Create alert
DELETE	`/api/alerts/:id`	Delete alert

MongoDB Schema

```
// Location
{ userId, city, country, lat, lon, isDefault }

// Settings
{ userId, temperatureUnit: 'celsius'|'fahrenheit',
  windSpeedUnit, theme }

// Alert
```

```
{ userId, locationId, type: 'rain'|'temperature'|'wind',
  condition, threshold, isActive }
```

Project 25: Password Manager 🔴

Description

Securely store and manage passwords.

Features

- ✅ Master password authentication
- ✅ Add password entries
- ✅ Categories/folders
- ✅ Password generator
- ✅ Copy to clipboard
- ✅ Auto-lock after inactivity
- ✅ Search passwords
- ✅ Password strength indicator
- ✅ Import/export (encrypted)
- ✅ Breach detection (HaveIBeenPwned)

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Encryption: crypto-js (client-side)
State: Zustand
Password: generate-password
Copy: react-copy-to-clipboard
```

Backend API

Method	Endpoint	Description
POST	`/api/auth/register`	Register with master password
POST	`/api/auth/login`	Login

Method	Endpoint	Description
POST	`/api/auth/verify-master`	Verify master password
GET	`/api/entries`	Get all entries (encrypted)
POST	`/api/entries`	Add entry
GET	`/api/entries/:id`	Get entry
PUT	`/api/entries/:id`	Update entry
DELETE	`/api/entries/:id`	Delete entry
GET	`/api/categories`	Get categories
POST	`/api/categories`	Create category
POST	`/api/export`	Export encrypted backup
POST	`/api/import`	Import backup
POST	`/api/breach-check`	Check if password breached

MongoDB Schema

```
// User
{ email, masterPasswordHash, salt, createdAt }

// Entry
{ userId, title, username, encryptedPassword, url,
  categoryId, notes, isFavorite, lastUsed, createdAt }

// Category
{ userId, name, icon }
```

Security Note: Passwords should be encrypted client-side before sending to server. Server never sees plaintext passwords.

Project 26: Real Estate Listing 🟡

Description

Property listings with search and filters.

Features

- ✅ List properties for sale/rent
- ✅ Property details (beds, baths, sqft)
- ✅ Image gallery
- ✅ Map location
- ✅ Search with filters
- ✅ Save favorite listings
- ✅ Contact agent form
- ✅ Mortgage calculator
- ✅ Similar properties
- ✅ Compare properties

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Maps: react-leaflet or @react-google-maps/api
Gallery: react-image-gallery
State: Zustand + @tanstack/react-query
Forms: react-hook-form + zod
```

Backend API

Method	Endpoint	Description
GET	`/api/properties`	Get listings (filters)
GET	`/api/properties/:id`	Get property
POST	`/api/properties`	Create listing
PUT	`/api/properties/:id`	Update listing
DELETE	`/api/properties/:id`	Delete listing
POST	`/api/properties/:id/images`	Upload images
POST	`/api/properties/:id/favorite`	Toggle favorite
GET	`/api/properties/:id/similar`	Get similar
POST	`/api/inquiries`	Send inquiry
GET	`/api/inquiries`	Get inquiries (for agent)

Method	Endpoint	Description
GET	`/api/favorites`	Get favorites

MongoDB Schema

```
// Property
{ userId, title, description, type: 'sale'|'rent',
  propertyType: 'house'|'apartment'|'condo',
  price, beds, baths, sqft, yearBuilt,
  address, city, state, zip, lat, lon,
  images: [], amenities: [], status, createdAt }

// Inquiry
{ propertyId, name, email, phone, message, createdAt }

// Favorite
{ userId, propertyId }
```

Project 27: Restaurant Menu & Ordering 🟡

Description

Digital menu with online ordering.

Features

- ✅ Menu with categories
- ✅ Item details (image, description, price)
- ✅ Add to cart
- ✅ Item customization (size, extras)
- ✅ Cart management
- ✅ Checkout (pickup/delivery)
- ✅ Order tracking
- ✅ Order history
- ✅ Special offers/coupons
- ✅ Admin: Manage menu/orders

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
State: Zustand
Forms: react-hook-form + zod
Animation: framer-motion
```

Backend API

Method	Endpoint	Description
GET	`/api/menu`	Get full menu
GET	`/api/menu/categories`	Get categories
GET	`/api/menu/items/:id`	Get item details
POST	`/api/admin/menu/items`	Add item
PUT	`/api/admin/menu/items/:id`	Update item
DELETE	`/api/admin/menu/items/:id`	Delete item
POST	`/api/orders`	Create order
GET	`/api/orders/:id`	Get order status
GET	`/api/orders`	Get user's orders
PATCH	`/api/admin/orders/:id/status`	Update status
GET	`/api/admin/orders`	Get all orders
GET	`/api/coupons/:code`	Validate coupon

MongoDB Schema

```
// Category
{ name, description, order, isActive }

// MenuItem
{ categoryId, name, description, price, image,
  customizations: [{ name, options: [{ name, price }], required }],
  isAvailable, isPopular, createdAt }

// Order
{ userId, items: [{ itemId, name, quantity, price,
  customizations: [] }], subtotal, tax, deliveryFee,
```

```
    discount, total, type: 'pickup'|'delivery',
    address, phone, status, paymentStatus, createdAt }

  // Coupon
  { code, discountType, discountValue, minOrder,
    maxUses, usedCount, expiresAt, isActive }
```

Project 28: Event Management 🟡

Description

Create and manage events with ticketing.

Features

- ✅ Create events (title, date, venue)
- ✅ Event details page
- ✅ Ticket types and pricing
- ✅ Registration/booking
- ✅ Attendee management
- ✅ QR code tickets
- ✅ Check-in system
- ✅ Event calendar
- ✅ Email notifications
- ✅ Public event listing

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Calendar: react-big-calendar
QR Code: qrcode.react
Scanner: html5-qrcode
State: Zustand + @tanstack/react-query
Forms: react-hook-form + zod
```

Backend API

Method	Endpoint	Description
GET	`/api/events`	Get user's events
POST	`/api/events`	Create event
GET	`/api/events/:id`	Get event
PUT	`/api/events/:id`	Update event
DELETE	`/api/events/:id`	Delete event
GET	`/api/public/events`	Browse events
GET	`/api/public/events/:id`	Event details
POST	`/api/events/:id/tickets`	Create ticket type
GET	`/api/events/:id/attendees`	Get attendees
POST	`/api/public/events/:id/register`	Register
GET	`/api/registrations/:id`	Get registration
GET	`/api/registrations/:id/ticket`	Get ticket QR
POST	`/api/events/:id/checkin`	Check in attendee
GET	`/api/events/:id/stats`	Event statistics

MongoDB Schema

```
// Event
{ userId, title, description, date, endDate, venue,
  address, coverImage, isPublic, maxAttendees,
  ticketTypes: [{ name, price, quantity, sold }],
  status: 'draft'|'published'|'cancelled', createdAt }

// Registration
{ eventId, userId, name, email, phone, ticketType,
  ticketCode, isCheckedIn, checkedInAt, createdAt }
```

Project 29: Portfolio Builder 🟡

Description

Create a personal portfolio website.

Features

- ✅ Choose template
- ✅ Edit personal info
- ✅ Add projects with images
- ✅ Skills section
- ✅ Experience timeline
- ✅ Education section
- ✅ Contact form
- ✅ Custom domain (subdomain)
- ✅ SEO settings
- ✅ Analytics (views)

Frontend Stack

```
Framework: React + Vite (editor) + Static export
UI Library: shadcn/ui
Styling: Tailwind CSS
Image Upload: react-dropzone
Rich Text: tiptap
State: Zustand
DnD: @dnd-kit/core
```

Backend API

Method	Endpoint	Description
GET	`/api/portfolio`	Get user's portfolio
PUT	`/api/portfolio`	Update portfolio
POST	`/api/portfolio/publish`	Publish portfolio
POST	`/api/upload/image`	Upload image
GET	`/api/projects`	Get projects
POST	`/api/projects`	Add project
PUT	`/api/projects/:id`	Update project
DELETE	`/api/projects/:id`	Delete project
PATCH	`/api/projects/reorder`	Reorder projects
POST	`/api/portfolio/contact`	Submit contact (public)

Method	Endpoint	Description
GET	`/api/portfolio/messages`	Get messages
GET	`/api/portfolio/analytics`	Get view stats
GET	`/api/public/:username`	Get public portfolio

MongoDB Schema

```
// Portfolio
{ userId, username, template, isPublished,
  personalInfo: { name, title, bio, avatar, resume },
  social: { github, linkedin, twitter, website },
  skills: [{ name, level }],
  experience: [{ company, position, startDate, endDate, description }],
  education: [{ school, degree, field, year }],
  seo: { title, description }, createdAt }

// Project
{ userId, title, description, image, tags: [],
  liveUrl, githubUrl, order }

// ContactMessage
{ portfolioId, name, email, message, createdAt }

// PageView
{ portfolioId, date, count }
```

Project 30: Inventory Management 🟡

Description

Track inventory and stock levels.

Features

- ✅ Add products with SKU
- ✅ Categories
- ✅ Stock quantity tracking
- ✅ Low stock alerts

- ✅ Stock in/out transactions
- ✅ Barcode support
- ✅ Suppliers management
- ✅ Purchase orders
- ✅ Reports and analytics
- ✅ Export data

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Barcode: react-barcode, html5-qrcode (scanner)
Charts: recharts
State: Zustand + @tanstack/react-query
Forms: react-hook-form + zod
Export: file-saver, xlsx
```

Backend API

Method	Endpoint	Description
GET	`/api/products`	Get products
POST	`/api/products`	Add product
GET	`/api/products/:id`	Get product
PUT	`/api/products/:id`	Update product
DELETE	`/api/products/:id`	Delete product
GET	`/api/products/low-stock`	Get low stock items
POST	`/api/transactions`	Record stock in/out
GET	`/api/transactions`	Get transactions
GET	`/api/categories`	Get categories
POST	`/api/categories`	Add category
GET	`/api/suppliers`	Get suppliers
POST	`/api/suppliers`	Add supplier
GET	`/api/purchase-orders`	Get POs
POST	`/api/purchase-orders`	Create PO

Method	Endpoint	Description
PATCH	`/api/purchase-orders/:id/receive`	Receive PO
GET	`/api/reports/inventory`	Inventory report
GET	`/api/export`	Export data

MongoDB Schema

```
// Product
{ userId, sku, barcode, name, description, category,
  quantity, minQuantity, costPrice, sellingPrice,
  supplierId, image, location, createdAt }

// Transaction
{ productId, userId, type: 'in'|'out', quantity,
  previousQty, newQty, reason, reference, createdAt }

// Supplier
{ userId, name, email, phone, address, notes }

// PurchaseOrder
{ userId, supplierId, items: [{ productId, quantity,
  unitPrice }], totalAmount, status: 'pending'|'received',
  orderDate, receivedDate }
```

Project 31: Learning Management System (Mini) 🔴

Description

Create and take courses.

Features

- ✅ Create courses with modules
- ✅ Video lessons (embed YouTube/Vimeo)
- ✅ Text/article lessons
- ✅ Quiz per module
- ✅ Progress tracking
- ✅ Completion certificate

- ✅ Course catalog
- ✅ Enrollment
- ✅ Instructor dashboard
- ✅ Student progress view

Frontend Stack

```
Framework: Next.js 14
UI Library: shadcn/ui
Styling: Tailwind CSS
Video: react-player
Rich Text: tiptap
Progress: Custom
State: Zustand + @tanstack/react-query
PDF: @react-pdf/renderer
```

Backend API

Method	Endpoint	Description
GET	`/api/courses`	Get published courses
GET	`/api/courses/:id`	Get course details
POST	`/api/instructor/courses`	Create course
PUT	`/api/instructor/courses/:id`	Update course
DELETE	`/api/instructor/courses/:id`	Delete course
PATCH	`/api/instructor/courses/:id/publish`	Publish
POST	`/api/instructor/courses/:id/modules`	Add module
PUT	`/api/modules/:id`	Update module
POST	`/api/modules/:id/lessons`	Add lesson
PUT	`/api/lessons/:id`	Update lesson
POST	`/api/courses/:id/enroll`	Enroll in course
GET	`/api/enrollments`	Get my enrollments
POST	`/api/lessons/:id/complete`	Mark complete
GET	`/api/enrollments/:id/progress`	Get progress
POST	`/api/quizzes/:id/submit`	Submit quiz

Method	Endpoint	Description
GET	`/api/enrollments/:id/certificate`	Get certificate

MongoDB Schema

```
// Course
{ instructorId, title, description, thumbnail,
  category, level, duration, isPublished,
  enrollmentCount, createdAt }

// Module
{ courseId, title, order }

// Lesson
{ moduleId, title, type: 'video'|'article'|'quiz',
  content, videoUrl, duration, order }

// Quiz
{ lessonId, questions: [{ question, options,
  correctAnswer }], passingScore }

// Enrollment
{ userId, courseId, enrolledAt, completedAt }

// Progress
{ enrollmentId, lessonId, isCompleted, completedAt }

// QuizAttempt
{ enrollmentId, quizId, score, passed, answers, completedAt }
```

Project 32: Team Chat Application 🔴

Description

Slack-like team messaging.

Features

- ✅ Workspaces/teams

- ✅ Channels (public/private)
- ✅ Direct messages
- ✅ Real-time messaging
- ✅ Message threads
- ✅ File sharing
- ✅ Emoji reactions
- ✅ @mentions
- ✅ Search messages
- ✅ Online status

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Real-time: socket.io-client
State: Zustand
File Upload: react-dropzone
Emoji: @emoji-mart/react
Rich Text: tiptap
```

Backend API

Method	Endpoint	Description
POST	`/api/workspaces`	Create workspace
GET	`/api/workspaces`	Get user's workspaces
GET	`/api/workspaces/:id`	Get workspace details
POST	`/api/workspaces/:id/invite`	Invite member
GET	`/api/workspaces/:id/channels`	Get channels
POST	`/api/channels`	Create channel
GET	`/api/channels/:id/messages`	Get messages
POST	`/api/channels/:id/messages`	Send message
GET	`/api/dm/:userId`	Get DM thread
POST	`/api/dm/:userId`	Send DM
GET	`/api/threads/:messageId`	Get thread
POST	`/api/threads/:messageId`	Reply to thread

Method	Endpoint	Description
POST	`/api/messages/:id/react`	Add reaction
POST	`/api/upload`	Upload file
GET	`/api/search?q=`	Search messages

WebSocket Events:

```
- message:new
- message:update
- message:delete
- typing:start
- typing:stop
- user:online
- user:offline
```

MongoDB Schema

```
// Workspace
{ name, ownerId, members: [{ userId, role }], createdAt }

// Channel
{ workspaceId, name, description, isPrivate,
  members: [], createdAt }

// Message
{ channelId, senderId, content, attachments: [],
  threadId, reactions: [{ emoji, userIds }],
  mentions: [], isEdited, createdAt }

// DirectMessage
{ participants: [userId], createdAt }
// DM messages stored in Message with different channelId type
```

Project 33: Product Review Platform 🟡

Description

Review and rate products.

Features

- ✅ Add products
- ✅ Write reviews with rating
- ✅ Pros/cons lists
- ✅ Photo reviews
- ✅ Helpful votes
- ✅ Filter by rating
- ✅ Sort reviews
- ✅ Review responses
- ✅ Report reviews
- ✅ Featured reviews

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Rating: Custom star component
Image: react-dropzone
State: Zustand + @tanstack/react-query
Forms: react-hook-form + zod
```

Backend API

Method	Endpoint	Description
GET	`/api/products`	Get products
GET	`/api/products/:id`	Get product with reviews
POST	`/api/admin/products`	Add product
PUT	`/api/admin/products/:id`	Update product
GET	`/api/products/:id/reviews`	Get reviews (filters)
POST	`/api/products/:id/reviews`	Add review
PUT	`/api/reviews/:id`	Update review
DELETE	`/api/reviews/:id`	Delete review
POST	`/api/reviews/:id/helpful`	Mark helpful
POST	`/api/reviews/:id/report`	Report review

Method	Endpoint	Description
POST	`/api/reviews/:id/respond`	Respond (seller)
GET	`/api/users/:id/reviews`	Get user's reviews
POST	`/api/upload/images`	Upload review images

MongoDB Schema

```
// Product
{ name, description, images: [], category, brand,
  avgRating, reviewCount, createdAt }

// Review
{ productId, userId, rating, title, content,
  pros: [], cons: [], images: [], helpfulCount,
  isVerifiedPurchase, response, respondedAt,
  isReported, createdAt }

// HelpfulVote
{ reviewId, userId }

// Report
{ reviewId, userId, reason, createdAt }
```

Project 34: Subscription Box Manager 🟡

Description

Manage subscription-based products/services.

Features

- ✅ Create subscription plans
- ✅ Customer subscriptions
- ✅ Billing cycles
- ✅ Pause/resume subscriptions
- ✅ Payment tracking (mock)
- ✅ Upcoming shipments
- ✅ Subscription history

- ✅ Promo codes
- ✅ Customer portal
- ✅ Admin dashboard

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
State: Zustand + @tanstack/react-query
Forms: react-hook-form + zod
Calendar: date-fns
Charts: recharts
```

Backend API

Method	Endpoint	Description
GET	`/api/plans`	Get subscription plans
POST	`/api/admin/plans`	Create plan
PUT	`/api/admin/plans/:id`	Update plan
DELETE	`/api/admin/plans/:id`	Delete plan
GET	`/api/subscriptions`	Get my subscriptions
POST	`/api/subscriptions`	Create subscription
GET	`/api/subscriptions/:id`	Get subscription
PATCH	`/api/subscriptions/:id/pause`	Pause
PATCH	`/api/subscriptions/:id/resume`	Resume
PATCH	`/api/subscriptions/:id/cancel`	Cancel
GET	`/api/admin/subscriptions`	All subscriptions
GET	`/api/payments`	Get payment history
POST	`/api/payments/:id/retry`	Retry payment
GET	`/api/shipments/upcoming`	Upcoming shipments
POST	`/api/coupons/validate`	Validate coupon
GET	`/api/admin/stats`	Dashboard stats

MongoDB Schema

```
// Plan
{ name, description, price, interval: 'monthly'|'quarterly'|'yearly',
  features: [], isActive, createdAt }

// Subscription
{ userId, planId, status: 'active'|'paused'|'cancelled',
  startDate, nextBillingDate, pausedAt, cancelledAt,
  shippingAddress, createdAt }

// Payment
{ subscriptionId, amount, status: 'paid'|'failed'|'pending',
  paymentDate, transactionId }

// Shipment
{ subscriptionId, trackingNumber, status, shippedDate,
  deliveredDate, items: [] }

// Coupon
{ code, discountType, discountValue, applicablePlans: [],
  maxUses, usedCount, expiresAt }
```

Project 35: Bug/Issue Tracker 🟡

Description

Track bugs and issues for projects.

Features

- ✅ Create projects
- ✅ Report issues
- ✅ Issue types (Bug, Feature, Task)
- ✅ Priority and severity
- ✅ Status workflow
- ✅ Assign to team members
- ✅ Comments and discussion
- ✅ Labels/tags
- ✅ File attachments

- ✅ Activity log
- ✅ Dashboard with metrics

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Rich Text: tiptap
State: Zustand + @tanstack/react-query
DnD: @dnd-kit/core
Charts: recharts
```

Backend API

Method	Endpoint	Description
GET	`/api/projects`	Get projects
POST	`/api/projects`	Create project
GET	`/api/projects/:id`	Get project
PUT	`/api/projects/:id`	Update project
POST	`/api/projects/:id/members`	Add member
GET	`/api/projects/:id/issues`	Get issues
POST	`/api/projects/:id/issues`	Create issue
GET	`/api/issues/:id`	Get issue
PUT	`/api/issues/:id`	Update issue
DELETE	`/api/issues/:id`	Delete issue
PATCH	`/api/issues/:id/status`	Update status
PATCH	`/api/issues/:id/assign`	Assign issue
POST	`/api/issues/:id/comments`	Add comment
GET	`/api/issues/:id/activity`	Get activity
POST	`/api/upload/attachment`	Upload file
GET	`/api/projects/:id/stats`	Get metrics

MongoDB Schema

```
// Project
{ ownerId, name, description, key, members: [{ userId, role }],
  labels: [{ name, color }], createdAt }

// Issue
{ projectId, issueNumber, type: 'bug'|'feature'|'task',
  title, description, status: 'open'|'in-progress'|'resolved'|'closed',
  priority: 'low'|'medium'|'high'|'critical',
  assigneeId, reporterId, labels: [], attachments: [],
  dueDate, createdAt, updatedAt }

// Comment
{ issueId, userId, content, createdAt }

// Activity
{ issueId, userId, action, field, oldValue, newValue, createdAt }
```

Project 36: Fitness Class Booking 🟡

Description

Book fitness classes at a gym/studio.

Features

- ✅ Class schedule calendar
- ✅ Class types (Yoga, HIIT, etc.)
- ✅ Instructor profiles
- ✅ Book/cancel classes
- ✅ Waitlist management
- ✅ Class capacity limits
- ✅ Booking history
- ✅ Membership management
- ✅ Check-in system
- ✅ Class reminders

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Calendar: react-big-calendar
State: Zustand + @tanstack/react-query
Forms: react-hook-form
```

Backend API

Method	Endpoint	Description
GET	`/api/classes`	Get class schedule
GET	`/api/classes/:id`	Get class details
POST	`/api/admin/classes`	Create class
PUT	`/api/admin/classes/:id`	Update class
DELETE	`/api/admin/classes/:id`	Delete class
POST	`/api/classes/:id/book`	Book class
DELETE	`/api/bookings/:id`	Cancel booking
POST	`/api/classes/:id/waitlist`	Join waitlist
GET	`/api/bookings`	Get my bookings
GET	`/api/instructors`	Get instructors
POST	`/api/admin/instructors`	Add instructor
POST	`/api/bookings/:id/checkin`	Check in
GET	`/api/memberships`	Get membership plans
GET	`/api/my-membership`	Get user membership

MongoDB Schema

```
// Class
{ title, type, instructorId, date, startTime, endTime,
  duration, capacity, bookedCount, waitlistCount,
  location, description, isRecurring, createdAt }

// Booking
{ classId, userId, status: 'booked'|'waitlist'|'cancelled'|'attended',
  bookedAt, checkedInAt }
```

```
// Instructor
{ name, bio, photo, specialties: [], socialLinks }

// ClassType
{ name, description, icon, color }

// Membership
{ userId, planId, status, startDate, endDate,
  classesRemaining }
```

Project 37: Recipe Sharing Community 🟡

Description

Share and discover recipes socially.

Features

- ✅ Post recipes
- ✅ Follow other users
- ✅ Feed of followed users' recipes
- ✅ Like recipes
- ✅ Comment on recipes
- ✅ Save to collections
- ✅ Recipe ratings
- ✅ Search and explore
- ✅ User profiles
- ✅ Trending recipes

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
State: Zustand + @tanstack/react-query
Image: react-dropzone
Forms: react-hook-form
Infinite Scroll: react-intersection-observer
```

Backend API

Method	Endpoint	Description
GET	`/api/feed`	Get feed (followed users)
GET	`/api/explore`	Explore recipes
GET	`/api/trending`	Trending recipes
POST	`/api/recipes`	Post recipe
GET	`/api/recipes/:id`	Get recipe
PUT	`/api/recipes/:id`	Update recipe
DELETE	`/api/recipes/:id`	Delete recipe
POST	`/api/recipes/:id/like`	Toggle like
POST	`/api/recipes/:id/rate`	Rate recipe
GET	`/api/recipes/:id/comments`	Get comments
POST	`/api/recipes/:id/comments`	Add comment
POST	`/api/recipes/:id/save`	Save to collection
GET	`/api/users/:id`	Get user profile
GET	`/api/users/:id/recipes`	Get user's recipes
POST	`/api/users/:id/follow`	Toggle follow
GET	`/api/collections`	Get my collections
POST	`/api/collections`	Create collection
GET	`/api/search?q=`	Search recipes

MongoDB Schema

```
// Recipe
{ userId, title, description, images: [],
  prepTime, cookTime, servings, difficulty,
  ingredients: [{ name, amount, unit }],
  instructions: [],  category, cuisine, tags: [],
  likeCount, commentCount, avgRating, createdAt }

// Like
{ recipeId, userId }
```

```
// Rating
{ recipeId, userId, value }

// Comment
{ recipeId, userId, content, createdAt }

// Follow
{ followerId, followingId }

// Collection
{ userId, name, recipeIds: [] }
```

Project 38: Podcast Directory 🟡

Description

Browse and track podcasts.

Features

- ✅ Browse podcast categories
- ✅ Search podcasts
- ✅ Podcast details with episodes
- ✅ Subscribe to podcasts
- ✅ Episode playback
- ✅ Playback history
- ✅ Create playlists
- ✅ Mark episodes played
- ✅ Download for offline
- ✅ Recommendations

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Audio: Custom player with Howler.js
State: Zustand (persist)
Infinite Scroll: react-intersection-observer
```

Backend API

Method	Endpoint	Description
GET	`/api/podcasts`	Browse/search podcasts
GET	`/api/podcasts/:id`	Get podcast details
GET	`/api/podcasts/:id/episodes`	Get episodes
POST	`/api/admin/podcasts`	Add podcast
POST	`/api/podcasts/:id/subscribe`	Toggle subscribe
GET	`/api/subscriptions`	Get subscriptions
GET	`/api/episodes/:id`	Get episode
POST	`/api/episodes/:id/played`	Mark played
GET	`/api/history`	Get play history
GET	`/api/playlists`	Get playlists
POST	`/api/playlists`	Create playlist
POST	`/api/playlists/:id/episodes`	Add to playlist
GET	`/api/categories`	Get categories
GET	`/api/recommendations`	Get recommendations

MongoDB Schema

```
// Podcast
{ title, author, description, image, feedUrl,
  category, language, episodeCount, subscriberCount,
  latestEpisodeDate, createdAt }

// Episode
{ podcastId, title, description, audioUrl, duration,
  publishedDate, episodeNumber, season }

// Subscription
{ userId, podcastId, subscribedAt }

// PlayHistory
{ userId, episodeId, progress, duration, playedAt }
```

```
// Playlist
{ userId, name, episodeIds: [] }
```

Project 39: Gift Registry 🟡

Description

Create and share gift wishlists.

Features

- ✅ Create registries (Wedding, Birthday, etc.)
- ✅ Add items with external links
- ✅ Price and quantity
- ✅ Mark as purchased
- ✅ Share registry link
- ✅ Privacy settings
- ✅ Thank you notes
- ✅ Gift tracking
- ✅ Contributor names
- ✅ Cash fund option

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
State: Zustand + @tanstack/react-query
Forms: react-hook-form + zod
Share: react-share
```

Backend API

Method	Endpoint	Description
GET	`/api/registries`	Get my registries
POST	`/api/registries`	Create registry
GET	`/api/registries/:id`	Get registry

Method	Endpoint	Description
PUT	`/api/registries/:id`	Update registry
DELETE	`/api/registries/:id`	Delete registry
POST	`/api/registries/:id/items`	Add item
PUT	`/api/items/:id`	Update item
DELETE	`/api/items/:id`	Remove item
GET	`/api/public/registries/:shareId`	Get public registry
POST	`/api/public/registries/:shareId/purchase`	Mark purchased
GET	`/api/registries/:id/purchases`	Get purchases
POST	`/api/registries/:id/thankyou`	Send thank you
POST	`/api/registries/:id/cashfund`	Contribute to fund

MongoDB Schema

```
// Registry
{ userId, title, type, eventDate, description,
  coverImage, shareId, isPublic, cashFundEnabled,
  cashFundGoal, cashFundTotal, createdAt }

// Item
{ registryId, name, description, image, url, price,
  quantity, purchasedQty, priority, category }

// Purchase
{ itemId, registryId, buyerName, buyerEmail, quantity,
  message, isAnonymous, purchasedAt }

// ThankYou
{ purchaseId, message, sentAt }
```

Project 40: Garage Sale / Marketplace 🟡

Description

List and browse items for sale.

Features

- ✅ List items with photos
- ✅ Categories
- ✅ Price and condition
- ✅ Location-based search
- ✅ Message seller
- ✅ Mark as sold
- ✅ Save favorites
- ✅ Price alerts
- ✅ Seller ratings
- ✅ Report listings

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Maps: react-leaflet
Image: react-dropzone
State: Zustand + @tanstack/react-query
Forms: react-hook-form + zod
```

Backend API

Method	Endpoint	Description
GET	`/api/listings`	Get listings (filters)
GET	`/api/listings/:id`	Get listing
POST	`/api/listings`	Create listing
PUT	`/api/listings/:id`	Update listing
DELETE	`/api/listings/:id`	Delete listing
PATCH	`/api/listings/:id/sold`	Mark sold
POST	`/api/listings/:id/favorite`	Toggle favorite
GET	`/api/favorites`	Get favorites
POST	`/api/messages`	Send message
GET	`/api/conversations`	Get conversations

Method	Endpoint	Description
GET	`/api/conversations/:id`	Get thread
POST	`/api/listings/:id/report`	Report listing
POST	`/api/users/:id/rate`	Rate seller
GET	`/api/users/:id/listings`	Get seller listings
GET	`/api/my-listings`	Get my listings
POST	`/api/upload/images`	Upload images

MongoDB Schema

```
// Listing
{ userId, title, description, price, images: [],
  category, condition: 'new'|'like-new'|'good'|'fair',
  location: { city, lat, lon }, status: 'active'|'sold',
  viewCount, favoriteCount, createdAt }

// Message
{ conversationId, senderId, content, createdAt }

// Conversation
{ listingId, buyerId, sellerId, lastMessage, updatedAt }

// Favorite
{ userId, listingId }

// Report
{ listingId, userId, reason, createdAt }

// Rating
{ sellerId, buyerId, rating, review, createdAt }
```

Project 41: Volunteer Management 🟡

Description

Coordinate volunteer activities.

Features

- ✅ Create volunteer opportunities
- ✅ Opportunity details and requirements
- ✅ Sign up for opportunities
- ✅ Volunteer hours tracking
- ✅ Calendar view
- ✅ Check-in/check-out
- ✅ Volunteer profiles
- ✅ Hour reports
- ✅ Certificates
- ✅ Team management

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Calendar: react-big-calendar
State: Zustand + @tanstack/react-query
Forms: react-hook-form + zod
PDF: @react-pdf/renderer
```

Backend API

Method	Endpoint	Description
GET	`/api/opportunities`	Get opportunities
POST	`/api/opportunities`	Create opportunity
GET	`/api/opportunities/:id`	Get opportunity
PUT	`/api/opportunities/:id`	Update opportunity
DELETE	`/api/opportunities/:id`	Delete opportunity
POST	`/api/opportunities/:id/signup`	Sign up
DELETE	`/api/opportunities/:id/signup`	Cancel signup
GET	`/api/opportunities/:id/volunteers`	Get signed up
POST	`/api/signups/:id/checkin`	Check in
POST	`/api/signups/:id/checkout`	Check out

Method	Endpoint	Description
GET	`/api/my-signups`	Get my signups
GET	`/api/my-hours`	Get my hours
GET	`/api/volunteers`	Get all volunteers
GET	`/api/volunteers/:id`	Get volunteer profile
GET	`/api/reports/hours`	Hours report
GET	`/api/certificates/:id`	Generate certificate

MongoDB Schema

```
// Opportunity
{ organizerId, title, description, date, startTime,
  endTime, location, slotsTotal, slotsRemaining,
  requirements, skills: [], category, createdAt }

// Signup
{ opportunityId, userId, status: 'registered'|'attended'|'no-show',
  checkinTime, checkoutTime, hoursLogged }

// Volunteer (User extended)
{ skills: [], interests: [], totalHours, bio }

// Certificate
{ userId, opportunityId, hours, issuedAt }
```

Project 42: Parking Spot Finder 🟡

Description

Find and reserve parking spots.

Features

- ✅ Map view of parking spots
- ✅ Filter by availability, price
- ✅ Parking spot details

- ✅ Reserve spot
- ✅ Time-based pricing
- ✅ Reservation history
- ✅ QR code entry/exit
- ✅ Add your parking spot
- ✅ Favorite locations
- ✅ Price comparison

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Maps: react-leaflet or @react-google-maps/api
QR: qrcode.react
State: Zustand + @tanstack/react-query
Date: date-fns
```

Backend API

Method	Endpoint	Description
GET	`/api/spots`	Get spots near location
GET	`/api/spots/:id`	Get spot details
POST	`/api/spots`	List parking spot
PUT	`/api/spots/:id`	Update spot
DELETE	`/api/spots/:id`	Delete spot
GET	`/api/spots/:id/availability`	Check availability
POST	`/api/reservations`	Make reservation
GET	`/api/reservations`	Get my reservations
GET	`/api/reservations/:id`	Get reservation
PATCH	`/api/reservations/:id/cancel`	Cancel
POST	`/api/reservations/:id/entry`	Record entry
POST	`/api/reservations/:id/exit`	Record exit
GET	`/api/my-spots`	Get my listed spots
GET	`/api/my-spots/:id/reservations`	Spot reservations

Method	Endpoint	Description
POST	`/api/spots/:id/favorite`	Toggle favorite

MongoDB Schema

```
// Spot
{ ownerId, name, address, lat, lon, images: [],
  pricePerHour, pricePerDay, availableFrom, availableTo,
  features: ['covered', 'ev-charging'], isActive }

// Reservation
{ spotId, userId, startTime, endTime, totalPrice,
  status: 'pending'|'confirmed'|'completed'|'cancelled',
  entryTime, exitTime, qrCode, createdAt }

// Favorite
{ userId, spotId }
```

Project 43: Time Tracker (Freelancer) 🟡

Description

Track time for client projects.

Features

- ✅ Start/stop timer
- ✅ Manual time entry
- ✅ Projects and tasks
- ✅ Client management
- ✅ Billable vs non-billable
- ✅ Daily/weekly timesheet
- ✅ Reports by project/client
- ✅ Export timesheets
- ✅ Invoicing integration
- ✅ Team time tracking

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Timer: Custom hook
Charts: recharts
State: Zustand (persist)
Export: file-saver
Calendar: date-fns
```

Backend API

Method	Endpoint	Description
GET	`/api/clients`	Get clients
POST	`/api/clients`	Create client
PUT	`/api/clients/:id`	Update client
DELETE	`/api/clients/:id`	Delete client
GET	`/api/projects`	Get projects
POST	`/api/projects`	Create project
PUT	`/api/projects/:id`	Update project
GET	`/api/tasks`	Get tasks
POST	`/api/tasks`	Create task
GET	`/api/time-entries`	Get time entries
POST	`/api/time-entries`	Create entry
PUT	`/api/time-entries/:id`	Update entry
DELETE	`/api/time-entries/:id`	Delete entry
POST	`/api/timer/start`	Start timer
POST	`/api/timer/stop`	Stop timer
GET	`/api/timer/running`	Get running timer
GET	`/api/reports/project/:id`	Project report
GET	`/api/reports/client/:id`	Client report
GET	`/api/reports/weekly`	Weekly report
GET	`/api/export`	Export timesheet

MongoDB Schema

```
// Client
{ userId, name, email, company, hourlyRate, color }

// Project
{ userId, clientId, name, description, hourlyRate,
  budget, status: 'active'|'completed'|'archived' }

// Task
{ projectId, name, isBillable }

// TimeEntry
{ userId, projectId, taskId, description, startTime,
  endTime, duration, isBillable, isRunning }
```

Project 44: Pet Tracker 🟢

Description

Track pet information and care.

Features

- ✅ Add pets with profiles
- ✅ Health records (vaccines, vet visits)
- ✅ Medication reminders
- ✅ Feeding schedule
- ✅ Weight tracking
- ✅ Photo gallery
- ✅ Pet documents storage
- ✅ Activity log
- ✅ Multiple pets
- ✅ Share with family

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
```

```
Styling: Tailwind CSS
Charts: recharts
Calendar: date-fns
Image: react-dropzone
State: Zustand
```

Backend API

Method	Endpoint	Description
GET	`/api/pets`	Get my pets
POST	`/api/pets`	Add pet
GET	`/api/pets/:id`	Get pet
PUT	`/api/pets/:id`	Update pet
DELETE	`/api/pets/:id`	Delete pet
POST	`/api/pets/:id/health-records`	Add record
GET	`/api/pets/:id/health-records`	Get records
POST	`/api/pets/:id/medications`	Add medication
GET	`/api/pets/:id/medications`	Get medications
POST	`/api/pets/:id/weight`	Log weight
GET	`/api/pets/:id/weight-history`	Get weight history
POST	`/api/pets/:id/photos`	Upload photo
GET	`/api/pets/:id/photos`	Get photos
POST	`/api/pets/:id/share`	Share with user
GET	`/api/reminders`	Get due reminders

MongoDB Schema

```
// Pet
{ userId, name, species, breed, birthday, gender,
  photo, microchipId, sharedWith: [] }

// HealthRecord
{ petId, type: 'vaccine'|'vet-visit'|'surgery'|'other',
  title, description, date, vetName, documents: [],
```

```
    nextDueDate }

  // Medication
  { petId, name, dosage, frequency, startDate, endDate,
    reminderTime, isActive }

  // WeightLog
  { petId, weight, unit, date }

  // Photo
  { petId, url, caption, date }
```

Project 45: Local Services Directory 🟡

Description

Find and review local services.

Features

- ✅ List services (plumber, electrician, etc.)
- ✅ Service categories
- ✅ Location-based search
- ✅ Service provider profiles
- ✅ Reviews and ratings
- ✅ Contact information
- ✅ Booking/inquiry form
- ✅ Verified providers
- ✅ Price range filter
- ✅ Featured listings

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Maps: react-leaflet
State: Zustand + @tanstack/react-query
Forms: react-hook-form + zod
```

Backend API

Method	Endpoint	Description
GET	`/api/services`	Browse/search services
GET	`/api/services/:id`	Get service details
POST	`/api/services`	List service (provider)
PUT	`/api/services/:id`	Update service
DELETE	`/api/services/:id`	Delete service
GET	`/api/services/:id/reviews`	Get reviews
POST	`/api/services/:id/reviews`	Add review
POST	`/api/services/:id/inquiry`	Send inquiry
GET	`/api/categories`	Get categories
GET	`/api/my-services`	Provider's services
GET	`/api/my-inquiries`	Provider's inquiries
PATCH	`/api/inquiries/:id/respond`	Respond to inquiry
POST	`/api/services/:id/favorite`	Toggle favorite
GET	`/api/favorites`	Get favorites

MongoDB Schema

```
// Service
{ providerId, name, category, description, images: [],
  address, city, lat, lon, phone, email, website,
  priceRange, availability, avgRating, reviewCount,
  isVerified, isFeatured, createdAt }

// Provider (User extended)
{ businessName, bio, experience, certifications: [] }

// Review
{ serviceId, userId, rating, title, content, createdAt }

// Inquiry
{ serviceId, userId, name, phone, email, message,
  status: 'pending'|'responded', response, createdAt }
```

Project 46: Document Collaboration 🔴

Description

Create and collaborate on documents.

Features

- ✅ Create documents
- ✅ Rich text editing
- ✅ Real-time collaboration
- ✅ Version history
- ✅ Comments
- ✅ Share with permissions
- ✅ Folder organization
- ✅ Templates
- ✅ Export (PDF, Word)
- ✅ Search documents

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Editor: tiptap with Collaboration extension
Real-time: socket.io-client + yjs
State: Zustand
Export: @react-pdf/renderer
```

Backend API

Method	Endpoint	Description
GET	`/api/documents`	Get my documents
POST	`/api/documents`	Create document
GET	`/api/documents/:id`	Get document
PUT	`/api/documents/:id`	Update document

Method	Endpoint	Description
DELETE	`/api/documents/:id`	Delete document
GET	`/api/documents/:id/versions`	Get versions
POST	`/api/documents/:id/versions`	Save version
GET	`/api/documents/:id/versions/:vId`	Get version
POST	`/api/documents/:id/share`	Share document
GET	`/api/shared-with-me`	Get shared docs
PATCH	`/api/documents/:id/permissions/:userId`	Update permission
POST	`/api/documents/:id/comments`	Add comment
GET	`/api/documents/:id/comments`	Get comments
GET	`/api/folders`	Get folders
POST	`/api/folders`	Create folder
GET	`/api/templates`	Get templates
POST	`/api/documents/:id/export`	Export document

WebSocket Events:

```
- document:update
- cursor:move
- user:join
- user:leave
```

MongoDB Schema

```
// Document
{ ownerId, title, content, folderId,
  sharedWith: [{ userId, permission: 'view'|'edit' }],
  isPublic, createdAt, updatedAt }

// Version
{ documentId, content, createdBy, createdAt }

// Comment
{ documentId, userId, content, position, resolved, createdAt }

// Folder
```

```
  { userId, name, parentId }

  // Template
  { name, content, category, isPublic }
```

Project 47: Meal Prep Planner 🟡

Description

Plan weekly meals and generate shopping lists.

Features

- ✅ Weekly meal calendar
- ✅ Recipe library
- ✅ Drag recipes to days
- ✅ Auto-generate shopping list
- ✅ Serving adjustments
- ✅ Nutritional summary
- ✅ Leftover tracking
- ✅ Save meal plans
- ✅ Share meal plans
- ✅ Copy previous week

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
DnD: @dnd-kit/core
Calendar: Custom week view
State: Zustand + @tanstack/react-query
```

Backend API

Method	Endpoint	Description
GET	`/api/recipes`	Get recipes
POST	`/api/recipes`	Add recipe

Method	Endpoint	Description
GET	`/api/recipes/:id`	Get recipe
PUT	`/api/recipes/:id`	Update recipe
DELETE	`/api/recipes/:id`	Delete recipe
GET	`/api/meal-plans`	Get meal plans
GET	`/api/meal-plans/week/:date`	Get week's plan
POST	`/api/meal-plans`	Add meal to plan
PUT	`/api/meal-plans/:id`	Update meal
DELETE	`/api/meal-plans/:id`	Remove meal
POST	`/api/meal-plans/copy`	Copy week
GET	`/api/shopping-list/:date`	Generate list
POST	`/api/shopping-list/items`	Add custom item
PATCH	`/api/shopping-list/items/:id`	Toggle checked
GET	`/api/saved-plans`	Get saved plans
POST	`/api/saved-plans`	Save current week
POST	`/api/saved-plans/:id/apply`	Apply saved plan

MongoDB Schema

```
// Recipe
{ userId, name, servings, prepTime, cookTime,
  ingredients: [{ name, amount, unit, category }],
  instructions: [], image, tags: [],
  nutrition: { calories, protein, carbs, fat } }

// MealPlan
{ userId, recipeId, date, mealType: 'breakfast'|'lunch'|'dinner'|'snack',
  servings, notes }

// SavedPlan
{ userId, name, meals: [{ dayOfWeek, mealType, recipeId, servings }] }

// ShoppingItem
{ userId, weekStart, name, amount, unit, category,
  recipeId, isChecked }
```

Project 48: Mood Tracker 🟢

Description

Track daily mood and emotions.

Features

- ✅ Log mood (1-5 scale or emojis)
- ✅ Add notes/journal
- ✅ Track activities
- ✅ Mood calendar view
- ✅ Monthly/weekly trends
- ✅ Correlate mood with activities
- ✅ Streak tracking
- ✅ Reminders
- ✅ Export data
- ✅ Insights

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Charts: recharts
Calendar: react-calendar-heatmap
State: Zustand
Animation: framer-motion
```

Backend API

Method	Endpoint	Description
GET	`/api/entries`	Get mood entries
POST	`/api/entries`	Log mood
GET	`/api/entries/:id`	Get entry
PUT	`/api/entries/:id`	Update entry

Method	Endpoint	Description
DELETE	`/api/entries/:id`	Delete entry
GET	`/api/entries/date/:date`	Get by date
GET	`/api/activities`	Get activity list
POST	`/api/activities`	Add custom activity
GET	`/api/stats/monthly/:year/:month`	Monthly stats
GET	`/api/stats/weekly`	Weekly stats
GET	`/api/stats/correlations`	Activity correlations
GET	`/api/stats/streaks`	Get streaks
GET	`/api/export`	Export data
GET	`/api/settings/reminder`	Get reminder
PUT	`/api/settings/reminder`	Set reminder

MongoDB Schema

```
// Entry
{ userId, date, mood, note, activities: [],
  sleep, weather, createdAt }

// Activity
{ userId, name, icon, isDefault }

// Settings
{ userId, reminderTime, reminderEnabled }
```

Project 49: Language Learning Vocabulary 🟡

Description

Build vocabulary with flashcards and quizzes.

Features

- ✅ Add words with translation

- ✅ Audio pronunciation
- ✅ Example sentences
- ✅ Word categories/lists
- ✅ Flashcard practice
- ✅ Spaced repetition
- ✅ Quiz mode
- ✅ Progress tracking
- ✅ Daily goals
- ✅ Streak tracking

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Audio: Web Speech API / custom
Animation: framer-motion
Charts: recharts
State: Zustand
```

Backend API

Method	Endpoint	Description
GET	`/api/words`	Get all words
POST	`/api/words`	Add word
GET	`/api/words/:id`	Get word
PUT	`/api/words/:id`	Update word
DELETE	`/api/words/:id`	Delete word
GET	`/api/lists`	Get word lists
POST	`/api/lists`	Create list
PUT	`/api/lists/:id`	Update list
DELETE	`/api/lists/:id`	Delete list
POST	`/api/lists/:id/words`	Add word to list
GET	`/api/practice/flashcards`	Get due cards
POST	`/api/practice/review`	Submit review

Method	Endpoint	Description
GET	`/api/quiz/generate`	Generate quiz
POST	`/api/quiz/submit`	Submit quiz
GET	`/api/stats`	Get learning stats
GET	`/api/goals`	Get daily goals
PUT	`/api/goals`	Set goals

MongoDB Schema

```
// Word
{ userId, word, translation, pronunciation,
  partOfSpeech, examples: [], audioUrl,
  listId, mastery, createdAt }

// List
{ userId, name, language, description }

// Review
{ wordId, userId, easeFactor, interval,
  dueDate, repetitions }

// QuizResult
{ userId, listId, score, totalQuestions,
  wrongAnswers: [], date }

// Goal
{ userId, dailyWordsTarget, dailyReviewTarget }

// Streak
{ userId, currentStreak, longestStreak, lastActiveDate }
```

Project 50: Home Maintenance Tracker 🟢

Description

Track home maintenance tasks and schedules.

Features

- ✅ Add maintenance tasks
- ✅ Set frequencies (monthly, yearly)
- ✅ Due date tracking
- ✅ Mark completed
- ✅ Maintenance history
- ✅ Cost tracking
- ✅ Service provider contacts
- ✅ Document storage
- ✅ Reminders
- ✅ Home inventory

Frontend Stack

```
Framework: React + Vite
UI Library: shadcn/ui
Styling: Tailwind CSS
Calendar: react-big-calendar
File Upload: react-dropzone
State: Zustand
Charts: recharts
```

Backend API

Method	Endpoint	Description
GET	`/api/tasks`	Get maintenance tasks
POST	`/api/tasks`	Create task
GET	`/api/tasks/:id`	Get task
PUT	`/api/tasks/:id`	Update task
DELETE	`/api/tasks/:id`	Delete task
POST	`/api/tasks/:id/complete`	Mark complete
GET	`/api/tasks/:id/history`	Get history
GET	`/api/tasks/upcoming`	Get upcoming
GET	`/api/tasks/overdue`	Get overdue
GET	`/api/contacts`	Get service contacts

Method	Endpoint	Description
POST	`/api/contacts`	Add contact
PUT	`/api/contacts/:id`	Update contact
DELETE	`/api/contacts/:id`	Delete contact
GET	`/api/inventory`	Get home inventory
POST	`/api/inventory`	Add item
PUT	`/api/inventory/:id`	Update item
POST	`/api/upload/document`	Upload document
GET	`/api/stats/costs`	Yearly costs

MongoDB Schema

```
// Task
{ userId, name, description, category, frequency,
  lastCompleted, nextDue, estimatedCost, assignedTo,
  documents: [], remindBefore, createdAt }

// CompletionLog
{ taskId, completedDate, notes, actualCost,
  serviceProviderId }

// ServiceContact
{ userId, name, company, phone, email, category,
  notes, rating }

// InventoryItem
{ userId, name, category, location, purchaseDate,
  warrantyExpiry, value, documents: [], notes }
```

📊 Project Summary by Category

Category	Projects	Count
Productivity	1, 3, 15, 17, 43	5
Finance	2, 10, 34	3
Health & Fitness	4, 9, 36, 44, 48	5

Category	Projects	Count
Food & Recipes	5, 27, 37, 47	4
Social & Community	7, 33, 37, 40	4
Organization	6, 8, 17, 18, 35	5
Learning	19, 23, 31, 49	4
Events & Scheduling	12, 21, 28, 41	4
E-commerce & Marketplace	26, 27, 39, 40	4
Utilities	13, 14, 20, 24, 25	5
Business	10, 22, 30, 43	4
Home & Life	42, 44, 45, 50	4

🚀 Recommended Build Order

Week 1-2: Warm Up (Beginner)

1. Task Manager (#1)
2. Note Taking App (#3)
3. Expense Tracker (#2)

Week 3-4: Level Up

4. Habit Tracker (#4)
5. Bookmark Manager (#6)
6. Pomodoro Timer (#15)

Week 5-6: Getting Confident

7. Job Application Tracker (#8)
8. Reading List (#16)
9. Flashcard App (#23)

Week 7-8: Intermediate

10. Recipe Manager (#5)
11. Invoice Generator (#10)

12. Quiz App (#19)

Week 9-10: More Features

13. Kanban Board (#17)
14. File Sharing (#20)
15. Contact Manager (#18)

Week 11-12: Advanced

16. Blog Platform (#7)
17. Appointment Scheduler (#21)
18. Event Management (#28)

Week 13-14: Complex Projects

19. Team Chat (#32)
20. LMS (#31)
21. Document Collaboration (#46)

🔧 Common Tech Stack

Frontend (All Projects)

```
{
  "framework": "React 18+ with Vite",
  "language": "TypeScript",
  "styling": "Tailwind CSS",
  "components": "shadcn/ui",
  "state": "Zustand",
  "data-fetching": "@tanstack/react-query",
  "forms": "react-hook-form + zod",
  "icons": "lucide-react"
}
```

Backend (All Projects)

```
{
  "runtime": "Node.js 18+",
  "framework": "Express.js",
  "database": "MongoDB with Mongoose",
  "auth": "JWT (jsonwebtoken + bcrypt)",
  "validation": "Joi or Zod",
  "file-upload": "Multer + Cloudinary",
  "email": "Nodemailer"
}
```

Common Backend Patterns

```
// Folder Structure
/src
  /controllers    // Route handlers
  /models         // Mongoose schemas
  /routes         // API routes
  /middleware     // Auth, validation, etc.
  /utils          // Helper functions
  /config         // Database, env config
  server.js       // Entry point

// Auth Middleware
const authMiddleware = (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  // Verify JWT and attach user to req
};

// Error Handling
const errorHandler = (err, req, res, next) => {
  res.status(err.status || 500).json({
    error: err.message || 'Server error'
  });
};
```

✅ Project Completion Checklist

For each project, ensure:

Frontend

- [ ] Responsive design (mobile, tablet, desktop)
- [ ] Loading states for all async operations
- [ ] Error handling with user-friendly messages
- [ ] Form validation with clear feedback
- [ ] Optimistic updates where appropriate
- [ ] Empty states for lists
- [ ] Keyboard navigation
- [ ] Dark/light theme (optional)

Backend

- [ ] Input validation on all endpoints
- [ ] Proper error responses with status codes
- [ ] Authentication where needed
- [ ] Authorization (users can only access their data)
- [ ] Pagination for list endpoints
- [ ] Rate limiting (optional)
- [ ] Request logging
- [ ] Environment variables for secrets

Deployment

- [ ] Frontend: Vercel or Netlify
- [ ] Backend: Railway, Render, or Fly.io
- [ ] Database: MongoDB Atlas
- [ ] Environment variables configured
- [ ] CORS properly configured

> **Pro Tip:** Start with the database schema design, then build the API, and finally the frontend. This approach ensures your data structure is solid before building the UI around it.

Happy Building! 🎉